

# Motorbiking boaters

---

## Motorbiking boaters (*student proposal*)

---

**Assessment:** ✓ Fit · **Category:** Games · **Assessed:** 2026-09-21 · **Status:** awaiting instructor approval

**Proposal (as submitted):** An endless side-scrolling freestyle jet-ski game: the player is an aspiring rider training for the BlueBull Jet-Ski Freeride Competition, chaining stunts and combos while staying on the jet ski as long as possible without wiping out. Tricks are performed with **phone interactions — swipes, taps, holds and tilts** — combined in sequences to trigger different stunts; simple tricks give small **time bonuses**, complex tricks/combos give more time and score multipliers, and failed tricks **subtract time** (a risk-and-reward loop). The course escalates with more obstacles and trick terrain the longer the player survives; the goal is high scores. **USP:** tricks are integral to survival — chain increasingly difficult stunts to extend game time in a high-risk / high-reward environment. Genre: casual arcade sports / endless runner. Platform: **Mobile, native Android**. Named technical features: gyroscope/accelerometer motion controls + gesture recognition; a **server-side gacha + user-accounts** solution; a **real-time physics engine** simulating the jet-ski + water interaction (arcade inertia, jumping above water); and **procedural endless level generation**. Named core-requirement notes: a C++ loop stable under heavy particle effects (splashes/sparks after a combo landing); tilt-to-move via gyro/accelerometer; a **custom level-testing tool** for designers to tweak procedural-level and player parameters without touching code. Design features: gesture-based freestyle trick system; a combo & scoring system; the risk-or-reward system; progressive difficulty & survival.

### *References:*

- [Alto's Odyssey Press Kit — Team Alto](#) (an endless side-scroller built on a one-touch trick system: chain combos, physics-based movement, procedurally generated terrain, escalating biomes — the endless-trick-survival + procgen precedent)

- [OlliOlli World on Steam](#) (a 2D side-scrolling trick game where you string tricks, grinds and spins into combos for score multipliers, landing cleanly or wiping out — the gesture-input trick-combo-scoring precedent)
- [Jetpack Joyride — Wikipedia](#) (the benchmark one-touch endless mobile arcade runner: procedurally varied hazards, coins, an in-run slot machine + a "Stash" of gadgets/cosmetics — the endless-mobile + gacha/economy precedent, incl. a "Wave Rider" jet-ski vehicle)

## Core Tech Requirements

Legend: ✓ Applies directly | ♦ Applies with domain adaptation | ● Optional | ✕ Not needed | » Second trimester

| Core Tech Requirement                                       | Motorbiking boaters |
|-------------------------------------------------------------|---------------------|
| External Database                                           | ✓                   |
| Authentication »                                            | ✓                   |
| Analytics »                                                 | ●                   |
| Real-time C++ Simulation Loop and/or C++ Performant Service | ✓                   |
| Application Communication                                   | ✓                   |
| Continuous Integration                                      | ✓                   |
| Data-Driven Architecture                                    | ✓                   |
| Front-end + Back-end                                        | ✓                   |
| Custom Tools Development                                    | ✓                   |
| 2D Graphical Output                                         | ✓                   |
| 3D (strong students only)                                   | ✕                   |
| Networking System                                           | ●                   |

### How each requirement applies:

*Platform fit:* **Main:** Native mobile (Native Android — the stated platform; device: Pixel 9a) — tilt-to-move and swipe/tap/hold trick input are native-touch/sensor-first; the Native PC build is the development/tools vehicle (the level-testing tool). **Extended:** a Linux server the team sets up by M2 (full use T2) hosting the gacha service + user accounts, plus an external database (Firebase or a server store) for accounts, cosmetics and high scores.

**Front end:** A side-scrolling 2D renderer (the rider + jet-ski, water surface, obstacles, trick terrain and escalating background) with the trick/combo/timer HUD and heavy splash/spark particle VFX, plus the gacha and score screens.

**Languages:** Native — Kotlin Native Android UI + the C++ engine via JNI (Android Studio); OpenGL ES drawn in a GLSurfaceView for the realtime 2D; Web/desktop — SDL2/3 + OpenGL ES for the PC dev/tools build.

**Back end:** The C++ engine core (fixed-timestep loop, arcade jet-ski + water-interaction physics with above-water inertia, gesture/gyro trick recognition, the combo/scoring/time economy, procedural endless generation and the particle systems) + a server-side gacha & user-accounts service. **Languages:** C++ (engine); the gacha/accounts service on the Linux server.

**Backend scope:** Local now, server-ready (T2) — the gacha + accounts + high-score seam set up & basic-tested on the Linux host by M2, full use in T2.

- *Loop and/or service:* the C++ loop is load-bearing — it runs the arcade jet-ski/water physics (buoyancy, above-water inertia, landing), the gesture/gyro trick recognition and combo resolution, the risk/reward time economy, the endless procedural scroll and the heavy splash/spark particle effects, holding a stable frame rate under a combo's particle burst.
- *Data-driven:* trick definitions and their gesture sequences, scoring/time-bonus values, obstacle/terrain pieces and their difficulty weights, player/physics parameters and the gacha cosmetic tables are all hot-reloadable data authored via the level-testing tool — designers tune without touching code.
- *2D:* side-scrolling sprite rendering + splash/spark particle VFX (custom-rendered). *3D:* not needed. *Networking:* no real-time multiplayer — the gacha/accounts/high-score calls are async client↔server (Application Communication ✓); a real-time Networking System is optional (●).
- *Custom tool:* the **level-testing tool** — tweak procedural-generation weights, obstacle/terrain sets and player/physics parameters with live preview, without recompiling.
- *External DB:* accounts, cosmetics/gacha inventory and high scores. *Auth »:* user accounts. *Analytics »:* optional — trick-attempt/failure and drop-off telemetry for difficulty tuning.

*Monetization note (design condition):* the **gacha** must be designed to the module's clean-monetization rule — **cosmetic-only, no pay-to-win, transparent**

**odds, no manipulative loops** (addressed in the M2 Design goal D8 below). A gacha that gated gameplay power or hid odds would fail that rule; keep it purely cosmetic.

## Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the Games pool in [milestone-goals-pool-per-category.pdf](#) (G1–G3 ★ common, G4+ picks).

*M1:*

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic**

Tech & dependencies

- **Tech: Web build:** SDL2/3 window + a fixed-timestep loop (C++17/20) running the endless scroll, reading swipe/tap/hold + gyro/accelerometer input, and resolving tricks/combos and the time economy; CMake, Git. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** None — foundation of this project.
- **Feeds into:** Every other goal; M1G5 physics and M1G7 procgen run on its tick.

- **G2 ★ Data-Driven Architecture of Core Objects / Entity System**

Tech & dependencies

- **Tech:** Entity/component registry in C++ with tricks + gesture sequences, scoring/time-bonus values, obstacle/terrain pieces, player/physics parameters and gacha cosmetic tables defined in JSON, hot-reloadable.
- **Prerequisites:** G1 (loop & entity update hooks).
- **Feeds into:** M2G2 (the level-testing tool edits this data), M1G7 (terrain pieces), M2G13 (trick/combo systems).

- **G3 ★ Debugging, Profiling, Stress Test & CI**

Tech & dependencies

- **Tech:** ImGui debug overlays (physics, trick state, combo timer), Tracy profiling to hold the frame budget under a **\*\*combo's particle burst\*\*** (splashes/sparks), Catch2 tests for trick recognition, scoring and data loading, gated by a GitHub Actions Windows+Android matrix with clang-format/clang-tidy.

- **Prerequisites:** G1 (loop to instrument).
- **Feeds into:** M2G3 (CI hardening); the frame budget M2G1 validates on-device.
- **G4 2D Graphics** — the side-scrolling renderer (rider, jet-ski, water, obstacles, trick terrain) is the whole read of the game.  
Tech & dependencies
  - **Tech: Web build:** Sprite/tile renderer with a scrolling camera, parallax background, a water surface and batching; SDL2/3/OpenGL ES, GLSL, stb\_image, FreeType HUD. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
  - **Prerequisites:** G1 (loop & window).
  - **Feeds into:** M2G4 (advanced water/particle VFX); every visible frame of the demo.
- **G5 Physics & Collision** — the arcade jet-ski + water-interaction physics (above-water inertia, jumping, landing) is the mechanical core.  
Tech & dependencies
  - **Tech:** C++ + GLM math, a custom arcade water/buoyancy + jet-ski model (thrust, jump arcs, landing angle affecting a clean-vs-wipeout outcome), and collision against obstacles/trick terrain.
  - **Prerequisites:** G1 (loop & tick), G2 (physics params).
  - **Feeds into:** The trick/landing feel; the risk/reward wipeout condition; M2G13 (combo scoring on landings).
- **G7 Procedural Level Generation** — the endless, escalating course (obstacles + trick terrain that ramp with survival time).  
Tech & dependencies
  - **Tech:** Seeded RNG assembling terrain/obstacle pieces from a weighted set, with a difficulty ramp driven by survival time and validation that a run stays completable.
  - **Prerequisites:** G2 (piece data model).
  - **Feeds into:** M2G14-class variety; the level-testing tool tunes the weights.

M2:

- **G1 ★ Architecture — Optimization & Platform Validation**

## Tech & dependencies

- **Tech: Web build:** Tracy profiling to hold the frame budget under peak splash/spark particle load, ASan on MSVC, memory/particle pools, and validation on-device (Pixel 9a, Native Android); the Linux server (gacha + accounts + high scores) set up (account + software + basic connectivity test) by M2. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
  - **Prerequisites:** All M1 goals (hardens the M1 codebase).
  - **Feeds into:** The Native-Android deliverable and a stable frame budget under combo bursts.
- **G2 ★ Content Editor — Core (Level-Testing Tool)** — the designer tool: tweak procedural-generation weights, obstacle/terrain sets and player/physics parameters with live preview, no code changes.

## Tech & dependencies

- **Tech:** Dear ImGui editor exposing procgen weights, terrain/obstacle sets, trick/scoring values and physics parameters as JSON, with round-trip serialization, live preview in the renderer and undo/redo.
- **Libraries/Software:** Dear ImGui, nlohmann/json, the project's SDL2/3/OpenGL ES renderer (live preview).
- **Prerequisites:** G2 (parameter data model), M1G4 (rendering for preview).
- **Feeds into:** Fast design iteration; the custom-tools showcase; procgen/difficulty tuning.

## • G3 ★ CI — CMake, Code Quality, Build Standards & Packaging

## Tech & dependencies

- **Tech: Web build:** CMake presets, clang-format + clang-tidy gates, reproducible Windows + Android Debug builds, tests (trick recognition, scoring, data loading) and packaging via GitHub Actions. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
  - **Prerequisites:** G3 (CI & test foundation).
  - **Feeds into:** Release/showcase builds of every goal.
- **G4 2D Graphics — Advanced** — the water and the **splash/spark particle VFX** (the game's core juice) matured, plus post-FX.



#### Tech & dependencies

- **Tech: Web build:** OpenGL/GLSL shaders for the water surface and spray, a pooled particle system for splashes/sparks/combo bursts, framebuffers and post-FX. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
  - **Prerequisites:** M1G4 (base renderer), M2G1 (optimization).
  - **Feeds into:** Showcase visuals; the tactile feedback of landing a combo.
- **G8 Firebase — Accounts, Leaderboards & Analytics** — user accounts, high-score leaderboards and the gacha's server-side cosmetic inventory.

#### Tech & dependencies

- **Tech:** Firebase Auth user accounts, Firestore for cosmetic/gacha inventory + high-score leaderboards, optional Analytics events, via the REST API from C++.
  - **Prerequisites:** G2 (profile/inventory/score data model).
  - **Feeds into:** Accounts + competitive high scores; the (clean) gacha cosmetics.
- **G13 Gameplay Systems II** — the **trick / combo / risk-reward scoring** system matured: gesture sequences → tricks, combo chaining, and the time-bonus / time-penalty economy.

#### Tech & dependencies

- **Tech:** C++ systems over the entity/data model: a gesture-sequence → trick resolver, combo chaining with multipliers, and the time economy (bonus on success, penalty on a failed/mistimed trick), all balancing-data-driven.
- **Prerequisites:** M1G2 (trick/scoring data), M1G5 (landing outcome feeds success/fail).
- **Feeds into:** The USP — chain-tricks-to-survive — at showcase; T2 trick expansion.

## Track Goals & Team Composition

**Recommended composition:** 6 CS + 1 Design + 2 Art (9) — the game reads by its **trick animation and water/particle VFX**, so two artists carry the rider + jet-ski, the many trick animations, the water/spray effects, the escalating environments and the gacha cosmetics; a single designer owns the gesture-trick vocabulary, the combo/scoring/risk-reward economy, the procedural-difficulty curve and the (clean) gacha economy (rules: min 1 Design mandated, max 9 total,  $D+A \leq 3$ ; see [team-formation.pdf](#)). If the trick/economy/gacha **systems** outgrow one designer, trade the 2nd artist for a 2nd designer. *Scope note (advisory):* arcade water physics + gesture tricks + combo/risk-reward + procedural endless + a server gacha + the editor is a large surface — treat single-run survival with the water physics, a handful of tricks + combos, the risk/reward timer and the level-testing tool as the M1/M2 floor, with the full trick set, escalating procgen and the server gacha as the stretch.

**Track goals** — suggested Design / UI-UX / Art / TEAM picks for this project; deliverables & quality ladders live in the track pool documents.

- **Design M1:**

- M1D1 ★ Features Design Document (the chain-tricks-to-survive risk/reward loop, the trick vocabulary and the escalating course; primary benchmark e.g. Alto's Odyssey or OlliOlli World, with a Singapore benchmark for the local-awareness criterion)
- M1D4 Game Feel & Touch Interaction Specification (the swipe/tap/hold/tilt trick gestures + gyro tilt-to-move, authored as tunable parameters — the differentiating input)
- M1D6 Systems & Economy Design (the trick→score→time economy, the combo multipliers, the risk/reward penalty and the cosmetic/gacha economy — every system with a purpose)

- **Design M2:**

- M2D1 ★ Features Design Iteration & Showcase Readiness
- M2D8 Monetization & Scope Integrity Design (the \*\*gacha reworked to the module's clean-monetization rule\*\* — cosmetic-only, transparent odds, no pay-to-win — plus a scope-integrity statement)

- **UI/UX M1** (graded by the Design instructor for the Games category):
  - M1U1 ★ UI Standards & Wireframe Baseline — readability standards for the timer/combo/score HUD and the gesture affordances, legible at speed on a phone.
- **UI/UX M2:**
  - M2U1 ★ Functional UI & Usability — HUD, results, gacha and account screens navigable unaided on the Native-Android target.
  - M2U2 ★ UX Validation & Final Polished UI — validated, polished HUD + trick feedback tested with casual players on the Pixel 9a.
- **Art M1:** 2 artists justified (trick-animation-heavy + water/particle VFX)
  - M1A1 ★ Art Bible / Proof of Concept (the arcade-sports visual direction — palette, water look, readability at speed)
  - M1A2 Character Concepts & Model Sheets (the rider + the jet-ski, with cosmetic-variant sheets)
  - M1A3 Environment, Props & VFX Concepts (the water, obstacles, trick terrain and the escalating biomes + splash/spark effect concepts)
  - M1A4 UI Art Concepts & Typography (timer/combo/score HUD, gacha screens)
  - M1A5 Animation Foundations (the trick animations — the core art payload — plus ride/land/wipeout)
  - M1A6 Early Asset Production — Prototype Support
- **Art M2:**
  - M2A1 ★ Critical Asset Production & In-Engine Visual Quality
  - M2A2 Character Animation — Extensive (the full trick set + combo/land/wipeout reactions)
  - M2A3 Environment Alive — Ambient Animation & Layering (living water + escalating background)
  - M2A4 UI Art — States & Feedback (trick-landed / failed / combo-multiplier states)
  - M2A5 VFX — Critical Effects & Transitions (splashes, spray, sparks, combo bursts — the game's core read)
  - M2A6 Cutscenes & Storyboards (the BlueBull competition intro / victory beats)

- **TEAM M1:**

- M1TA1 ★ Overall Audio Experience — initial BGM/SFX from the week-5 needs list present & working in the prototype (upbeat arcade BGM, engine/splash, trick-success/fail, combo and wipeout cues), suitable and correctly triggered.
- M1TA2 ★ Audio Engine Functionality — data-driven audio pipeline (add/replace files without recompiling) + reliable multi-channel playback for overlapping trick/splash SFX.

- **TEAM M2:**

- M2TA1 ★ Overall Audio Experience — standard Games floor (arcade BGM + engine, splash, trick-success/fail, combo, obstacle and wipeout SFX,  $\geq 6$  types), rising with the escalating difficulty.
- M2TA2 ★ Audio Engine Functionality — audio properties de/serialised and the full playback functionality the build needs (multi-channel mixing; a distinct combo-tier cue).
- M2TP1 ★ Final Presentation — 7 min in front of class + instructors: live custom-tools showcase (the level-testing tool), prototype demo, team post-mortem (wk14)

---

*Generated by the Project Proposal Assessor skill · grounding: live folder  
(Project 2 Technical adjustments/ ) · references verified 2026-09-21.*